Chapter Two

Fundamental of Algorithmic Problem Solving

Outline of the chapter

- Introduction
- Exact versus approximate solution
- Major classes of problems and corresponding algorithms
- Algorithm Design Approaches:
  - Divide and Conquer
  - Greedy approach
  - Dynamic programming approach
  - Branch and Bound approach

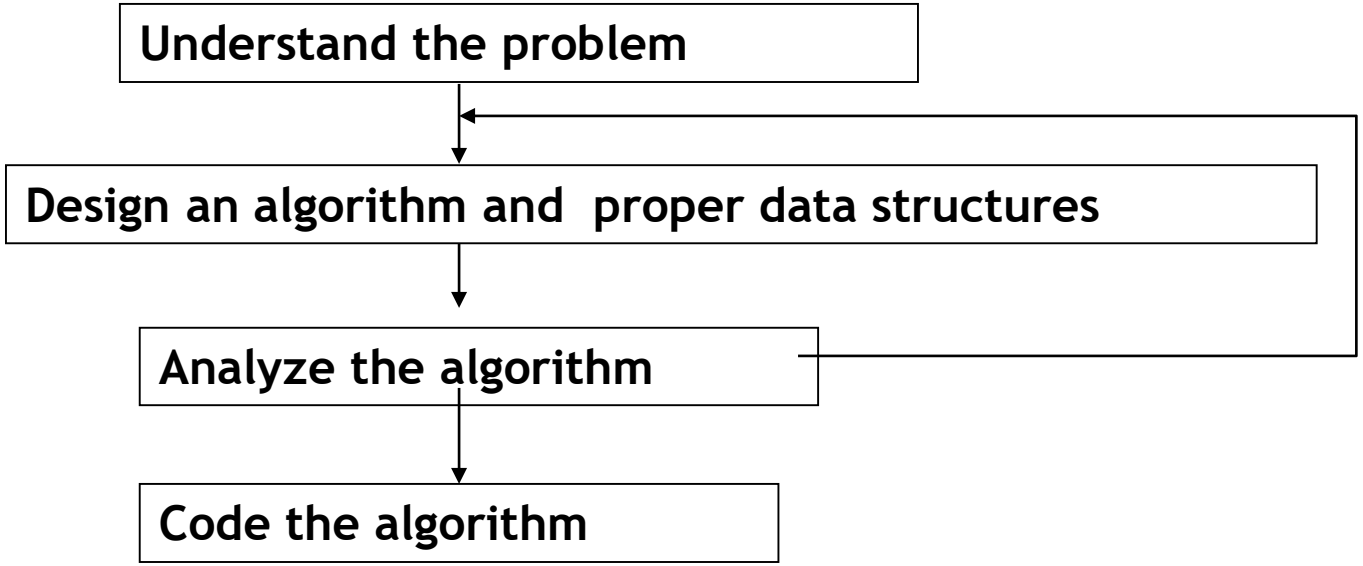
Introduction

- An algorithm is an exact specification of how to solve a computational problem
- An algorithm must specify every step completely, so a computer can implement it without any further "understanding"
- An algorithm must work for all possible inputs of the problem.
- Algorithms must be:
  - Correct: For each input produce an appropriate output
  - Efficient: run as quickly as possible, and use as little memory as possible - more about this later
- There can be many different algorithms for each computational problem.
- What is algorithm?
  - An algorithm is a sequence of unambiguous instructions for solving a problem, i.e., for obtaining a required output for any legitimate input in a finite amount of time.

Algorithmic Problem Solving

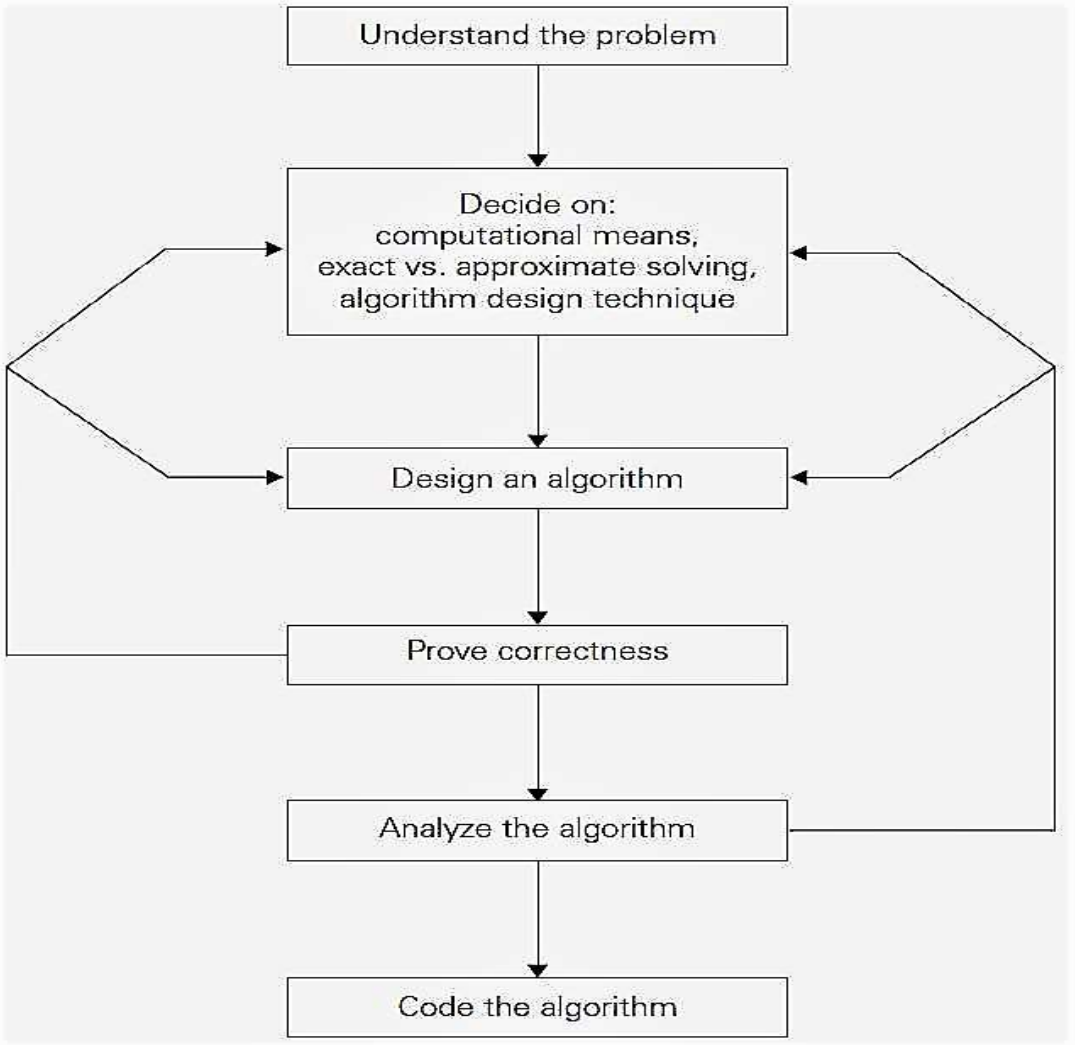
- Sequence of steps one typically goes through in designing and analyzing an algorithm
  - Understanding the Problem
  - Ascertaining the Capabilities of the Computational Device
  - Choosing between Exact and Approximate Problem Solving
  - Algorithm Design Techniques
  - Designing an Algorithm and Data Structures
  - Methods of Specifying an Algorithm
  - Proving an Algorithm's Correctness
  - Analyzing an Algorithm
  - Coding an Algorithm

Algorithmic Problem Solving



- Ascertaining the Capabilities of a Computational Device
- Choosing between Exact and Approximate Problem Solving
- Deciding on Appropriate Data Structures
- Algorithm Design
- Algorithm Analysis
- Coding

Algorithmic Problem Solving



1.Understanding the Problem

- Understand completely the problem given before design
- An input to an algorithm specifies an instance of the problem the algorithm solves.
- It is very important to specify exactly the set of instances the algorithm needs to handle.
- A correct algorithm is not one that works most of the time, but one that works correctly for all legitimate inputs.

2. Ascertaining the Capabilities of the Computational Device

- Ascertain the capabilities of the computational device the algorithm is intended for.
- Majority of the algorithms can be programmed for a computer closely resembling the von Neumann machine. (random-access machine (RAM) model)
- Algorithms designed to be executed on such machines are called sequential algorithms.
- Algorithms that take advantage of concurrent execution capability are called parallel algorithms.
- Should not worry about the speed and amount of memory of a computer at if you design an algorithm for a scientific purpose.

3. Choosing between Exact and Approximate Problem Solving

- An exact algorithm finds the solution to the problem asked. This is by contrast with an approximate algorithm, which only gets close to the solution. As others explain, there are practical situations such that exact algorithms cannot be used and one must content oneself with approximations.
- Choose between solving the problem exactly or solving it approximately.
- An algorithm is called an exact algorithm; in the latter case, an algorithm is called an approximation algorithm
- There are important problems that simply cannot be solved exactly for most of their instances.
  - Examples include extracting square roots, solving nonlinear equations, and evaluating definite integrals.
- Available algorithms for solving such problems exactly can be unacceptably slow.

4. Algorithm Design Techniques

- An algorithm design technique is a general approach to solving problems algorithmically
- They provide guidance for designing algorithms for new problems.
- Algorithm design techniques make it possible to classify algorithms according to an underlying design idea; (They can serve as a natural way to both categorize and study algorithms)
  - N.B. Algorithm Design Techniques are presented and described more latter on this chapter under [Algorithm Design Approaches subtitle]

5. Designing an Algorithm and Data Structures.

- Assigning an algorithm for a particular problem may be a challenging task even if you have a particular design technique.
- Sometimes, several techniques need to be combined.
- There are algorithms that are hard to pinpoint as applications of the known design techniques.
- Getting an algorithm often requires a nontrivial ingenuity on the part of the algorithm designer
- One should pay close attention to choosing data structures appropriate for the operations performed by the algorithm.

6. Methods of Specifying an Algorithm

- These are the two options that are most widely used nowadays for specifying algorithms.
- Pseudocode is a mixture of a natural language and programming language like constructs.
- specifying algorithms was a flowchart, a method of expressing an algorithm by a collection of connected geometric shapes containing descriptions of the algorithm's steps
- This representation technique has proved to be inconvenient for all but very simple algorithms;

7. Proving an Algorithm's Correctness

- Once an algorithm has been specified, you have to prove its correctness.
- That is, you have to prove that the algorithm yields a required result for every legitimate input in a finite amount of time.
  - For example, the correctness of Euclid's algorithm for computing the greatest common divisor stems from the correctness of the equality

$$\gcd(m, n) = \gcd(n, m \bmod n)$$

- For some algorithms, a proof of correctness is quite easy; for others, it can be quite complex.
- A common technique for proving correctness is to use mathematical induction

8. Analyzing an Algorithm

- Once an algorithm has been specified, you have to prove its correctness.
- After correctness, by far the most important is efficiency.
- Two kinds of algorithm efficiency: time efficiency, indicating how fast the algorithm runs, and space efficiency, indicating how much extra memory it uses
- Another desirable characteristic is simplicity.
- Simpler algorithms are easier to understand and easier to program; consequently, the resulting programs usually contain fewer bugs.
  - Example : Eclud's alg. vs The third alg.

8. Analyzing an Algorithm

- Yet another desirable characteristic of an algorithm is generality
- There are, in fact, two issues here: generality of the problem the algorithm solves and the set of inputs it accepts.
- On the first issue, note that it is sometimes easier to design an algorithm for a problem posed in more general terms. Eg. To check if two numbers are relatively prime numbers
- There are situations, however, where designing a more general algorithm is unnecessary or difficult or even impossible.
  - Ex: standard formula for roots of a quadratic equation cannot be generalized to handle polynomials of arbitrary degrees.

8. Analyzing an Algorithm

- As to the set of inputs, your main concern should be designing an algorithm that can handle a set of inputs that is natural for the problem at hand.
- For example, excluding integers equal to 1 as possible inputs for a greatest common divisor algorithm would be quite unnatural
- On the other hand, although the standard formula for the roots of a quadratic equation holds for complex coefficients, we would normally not implement it on this level of generality unless required

9. Coding an Algorithm

- Most algorithms are destined to be ultimately implemented as computer programs.
- However, there is a possibility of making the transition from an algorithm to a program either incorrectly or very inefficiently.
- The validity of programs can be established by either with full mathematical rigor or with testing.
- Testing of computer programs is an art rather than a science.
- Implementing an algorithm correctly is necessary but not sufficient: *Do not diminish your algorithm's power by an inefficient implementation.*
- Another important issue of algorithmic problem solving is the question of whether or not every problem can be solved by an algorithm.

Important Problem Types

- Problems one encounters in computing
  - Sorting
  - Searching
  - String processing (e.g. string matching)
  - Graph problems (e.g. graph coloring problem)
  - Combinatorial problems (e.g. maximizes a cost)
  - Geometric problems (e.g. convex hull problem)
  - Numerical problems (e.g. solving equations)

Algorithm Design Approaches

- We can choose from a wide range of algorithm design techniques. For insertion sort, we used an incremental approach: having sorted the sub array A[1….j-1], we inserted the single element A[j] into its proper place, yielding the sorted sub array A[1….j-1].
- Selecting a proper designing technique for an algorithm is the most difficult and important task. Most of the programming problems may have more than one solution.
- Algorithm design techniques give guidance and direction on how to create a new algorithm. Though there are literally thousands of algorithms, there are very few design techniques.
- The study of these techniques help us to categorize or organize the algorithms we know and in that way to understand them better.

Algorithm Design Approaches

- The following is a list of several popular design approaches:
  - Divide and conquer
  - Greedy Method
  - Dynamic Programming
  - Branch-and-Bound
  - Brute force
  - Backtracking

The Divide-and-Conquer Approach

- Many useful algorithms are recursive in structure: to solve a given problem, they call themselves recursively one or more times to deal with closely related sub problems.
- These algorithms typically follow a divide-and-conquer approach:
  - they break the problem into several sub problems that are similar to the original problem but smaller in size, solve the sub problems recursively, and
  - then combine these solutions to create a solution to the original problem.
- The divide-and-conquer paradigm involves three steps at each level of the recursion:
  - Divide the problem into a number of sub problems that are smaller instances of the same problem.
  - Conquer the sub problems by solving them recursively. If the sub problem sizes are small enough, however, just solve the sub problems in a straightforward manner.
  - Combine the solutions to the sub problems into the solution for the original problem.

Greedy Method Approach

- A Greedy algorithm is an approach to solving a problem that selects the most appropriate option based on the current situation. This algorithm ignores the fact that the current best result may not bring about the overall optimal result.
- Even if the initial decision was incorrect, the algorithm never reverses it.
- These are iterative algorithms that at each iteration the criterion used is to maximize some "gain" or some objective function.
  - The term "greedy" refers to the fact that the algorithms do this in a "short sighted" way; they try to maximize immediate gain, disregarding the big picture ("get the most I can get *now* ").
    - For this reason, they can fail to determine a global maximum or minimum (they could "fall in a trap" and converge to some local maximum)
- Dijkstra's, Prim's, and Kruskal's algorithms are all greedy algorithms

Dynamic Programming

- Dynamic Programming is an algorithm design technique for optimization problems: often minimizing or maximizing.
- Like divide and conquer, DP solves problems by combining solutions to subproblems.
- Unlike divide and conquer, subproblems are not independent.
  - Subproblems may share subsubproblems,
  - However, solution to one subproblem may not affect the solutions to other subproblems of the same problem. (More on this later.)
- DP reduces computation by
  - Solving subproblems in a bottom-up fashion.
  - Storing solution to a subproblem the first time it is solved.
  - Looking up the solution when subproblem is encountered again.
- Key: determine structure of optimal solutions

Steps in Dynamic Programming

1. Characterize structure of an optimal solution.
2. Define value of optimal solution recursively.
3. Compute optimal solution values either top-down with caching or bottom-up in a table.
4. Construct an optimal solution from computed values.

Branch-and-Bound Technique

- **Branch and bound** (**BB**, **B&B**, or **BnB**) is a method for solving optimization problems by breaking them down into smaller sub-problems and using a bounding function to eliminate sub-problems that cannot contain the optimal solution.
- It is an algorithm design paradigm for discrete and combinatorial optimization problems, as well as mathematical optimization.
- A branch-and-bound algorithm consists of a systematic enumeration of candidate solutions by means of state space search: the set of candidate solutions is thought of as forming a rooted tree with the full set at the root. The algorithm explores *branches* of this tree, which represent subsets of the solution set.
- Before enumerating the candidate solutions of a branch, the branch is checked against upper and lower estimated *bounds* on the optimal solution, and is discarded if it cannot produce a better solution than the best one found so far by the algorithm.
- The algorithm depends on efficient estimation of the lower and upper bounds of regions/branches of the search space. If no bounds are available, the algorithm degenerates to an exhaustive search.

Branch-and-Bound Technique

- Branch and bound is one of the techniques used for problem solving. It is similar to the backtracking since it also uses the state space tree. It is used for solving the optimization problems and minimization problems.
- If we have given a maximization problem then we can convert it using the Branch and bound technique by simply converting the problem into a maximization problem.
- The basic concept underlying the branch-and-bound technique is to *divide and conquer*.
- Since the original "large" problem is hard to solve directly, it is *divided* into smaller and smaller subproblems until these subproblems can be *conquered*.
  - The *dividing (branching)* is done by partitioning the entire set of feasible solutions into smaller and smaller subsets.
  - The *conquering (fathoming)* is done partially by
    - (i) giving a *bound* for the best solution in the subset;
    - (ii) discarding the subset if the bound indicates that it can't contain an optimal solution.
- These three basic steps
  - branching, bounding, and fathoming -